

Const Correctness & Its 4 Levels

Const Correctness is a promise you make to the compiler: "I will not mutate this data."

In a massive codebase where data is constantly being passed between functions, knowing exactly what is allowed to change and what is not prevents catastrophic side-effect bugs. Writing "**const correct**" code means applying the **const** keyword at every possible level of your architecture.

Level 1: Pass-by-Const-Reference

Passing by reference is dangerous because the function can accidentally mutate the original object. **The Fix:** You use **const &**. This gives you the speed of passing a pointer, with the safety of passing a copy.

```
// Bad: Slow deep copy
void printData(string data) { cout << data; }

// Bad: Fast, but dangerous (can accidentally change the original)
void printData(string& data) { cout << data; }

// PERFECT: Fast and completely safe
void printData(const string& data) { cout << data; }
// data = "new string"; // COMPILER ERROR! You promised not to change it.
```

Rule of thumb: If a function reads a complex object but doesn't modify it, always pass it by **const &**.

Level 2: The Pointer Confusion (Read it Right-to-Left)

Does the **const** protect the pointer itself, or the data the pointer is looking at? To solve this, use the "**Read it Right-to-Left**" Rule.

1. Pointer to Const Data

```
const int *ptr = &x;
```

- **Read right-to-left:** **ptr** is a pointer (*) to an **int** that is **const**.
- **Meaning:** The pointer can look at a new address, but it cannot change the data it is looking at.

```
ptr = &y;    // VALID
*ptr = 10;   // INVALID
```

2. Const Pointer

```
int* const ptr = &x;
```

- **Read right-to-left:** **ptr** is a **const** pointer (*) to an **int**.
- **Meaning:** The pointer is locked to one memory address, but the data inside it can be changed.

```
*ptr = 10;   // VALID
ptr = &y;     // INVALID
```

3. Const Pointer to Const Data

```
const int *const ptr = &x;
```

- **Read right-to-left:** ptr is a const pointer (*) to an int that is const.
- **Meaning:** Everything is locked. You can't move the pointer, and you can't change the data.

Level 3: Const Member Functions

When you design a Class, you must label any method that does not alter the object's internal state as a **const** method.

```
int getBalance() const { return balance; }
```

Why is this mandatory? If a function argument is **const BankAccount&**, the compiler will **only** allow you to call **const** methods on it.

```
void printStatement(const BankAccount &account) {  
    // If getBalance() isn't marked 'const', this line throws a compiler error!  
    cout << "Balance: " << account.getBalance();  
}
```

Level 4: The mutable Escape Hatch

Sometimes, a method is logically constant (it doesn't change the core data of the object), but technically needs to modify a hidden background variable. E.g., a mutex lock for thread safety, or a cached value.

To bypass the strict **const** rules, C++ gives you the **mutable** keyword. It tells the compiler: "Even if this object is locked down as const, allow this specific variable to be mutated."

```
class MathHeavyTask {  
private:  
    int data;  
    mutable int cache; // Allowed to change in const functions!  
    mutable bool isCached = false;  
  
public:  
    int calculateCost() const { // We promise not to change the core 'data'  
        if (isCached) { return cache; // Fast return }  
  
        // We can change these variables because they are marked 'mutable'.  
        cache = data * 9999 / 42;  
        isCached = true;  
  
        return cache;  
    }  
};
```